

Proflog: A Tableau-Based Logic Programming Kernel in miniKanren

ANONYMOUS AUTHOR(S)

We report on an implementation of Melvin Fitting’s Proflog, a logic programming language based on the tableau proof procedure. The system extends a first-order tableau prover with equality, written in Clojure’s `core.logic`, with Fitting’s Procedure Call rule. Formulae thereby acquire a programmatic character: evaluating a program is a matter of closing the relevant tableau(s). Following alphaLeanTAP, successful evaluation also produces an explicit proof trace. We describe the proof kernel, the surface-to-kernel translation, and worked executions of Fitting’s reference programs.

CCS Concepts: • **Theory of computation** → **Logic and verification**; *Automated reasoning*; • **Software and its engineering** → *Constraint and logic languages*.

Additional Key Words and Phrases: miniKanren, logic programming, tableaux, Proflog, proof search

ACM Reference Format:

Anonymous Author(s). 2026. Proflog: A Tableau-Based Logic Programming Kernel in miniKanren. In *Proceedings of The 2026 miniKanren and Relational Programming Workshop (miniKanren 2026)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 INTRODUCTION

Fitting proposed Proflog as an exploration in alternative bases for logic programming [3]. By comparison, Prolog operationalizes Horn clauses by resolution; Proflog instead manipulates formulae of first-order logic with equality by a tableau proof procedure, augmented with a mechanism for recursive procedure calls. The core tableau procedure employs standard branch closure, alpha/beta decomposition of connectives, and gamma/delta treatment of quantifiers. Equality is defined over a weak Herbrand setting, in which equal terms must be syntactically identical after branch substitutions are applied. The source code of a Proflog program is a series of clauses, each an implication between a formula ϕ and an atomic formula R :

$$R(x_1, \dots, x_n) \leftarrow \phi(x_1, \dots, x_n).$$

“Programmaticity” is introduced via the Procedure Call rule, which allows clauses to compose. During an attempt to close a tableau, a saved literal whose relation is defined by the program may open a subsidiary tableau for the corresponding clause body; the subsidiary proof then contributes to closure of the original branch. Figure 1 summarizes the implementation layers around this kernel rule.

This changes the usual programming/proving boundary. In Proflog, a program is not a collection of Horn clauses with procedural negation as failure. It is a finite collection of first-order definitions, at most one per relation symbol, whose queries are interpreted by two proof searches: a query succeeds when the negated query closes, fails when the query itself closes, and is unresolved when neither direction closes within the admitted search. This mirrors Fitting’s supervaluation semantics and makes some operational warnings visible, especially the warning that factoring a formula into an auxiliary relation may change the program.

The implementation reported here is written in Clojure and `core.logic` in the miniKanren tradition [2, 5]. Its immediate prover lineage is alphaLeanTAP, a relational tableau prover for first-order classical logic [4], and a Clojure/`core.logic` nominal implementation reference by Amin [1]. Proflog extends that lineage in the direction Fitting sketched: first-order proof search becomes a program evaluator, and program evaluation returns proof traces whose step tags record the tableau work performed.

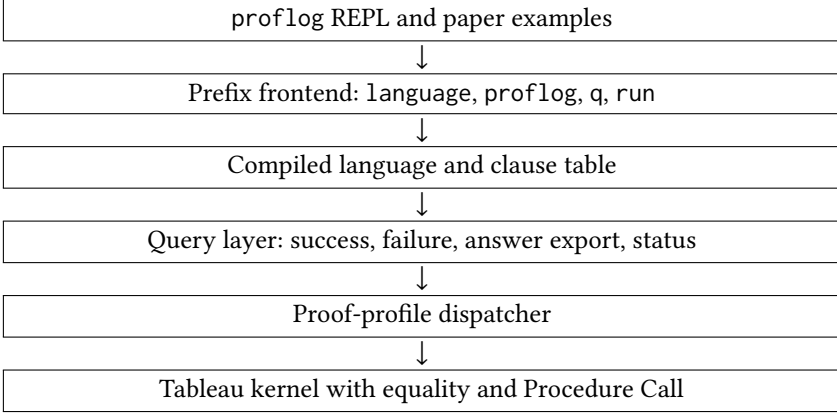


Fig. 1. Reviewer-visible Proflog layers from surface programs to the proof-producing kernel.

2 IMPLEMENTATION

2.1 Proof Kernel

The kernel is a relation over proof states. Its central entry point is `prove-stateo`; public calls use `kernel/prove` for direct formulae and `kernel/prove-program` for program-bearing formulae. A branch state contains the focused formula, pending formulae, saved literals, the lexical environment for bound variables, gamma-introduced proof variables, delta parameters, an explicit equality substitution, delayed disequalities, the compiled program, fuel, and the proof term being constructed.

The source does not package this state in a record; the slots are the argument vector of the kernel relation itself:

```

(defn prove-stateo
  [fml unexpanded lits env proof-vars
   sigma sigma-out neqs neqs-out
   prog gamma-terms fuel proof]
  ...)

```

The equality design is deliberately explicit. Positive equality extends a free-constructor substitution; negative equality is retained in a symbolic disequality store until later bindings force closure. This keeps equality, disequality, and procedure-call interaction visible in proof objects. A small closure proof, for example, has the shape:

```
(conj (savefml (close)))
```

An equality/disequality proof exposes the order of work:

```
(conj (neq-store (eq-step (eq-bind) (neq-close))))
```

Procedure calls add `pos-call` and `neg-call` proof steps. A positive call closes the relation body. A negative call closes the normalized negation of that body. Both calls run under the same kernel relation rather than through host-language evaluation of the source program.

2.2 Surface Syntax and Descent

The implemented frontend is a Clojure-readable prefix DSL that preserves Fitting's clause organization. The intended handwritten notation is:

```
p(x) :- x = a.
```

```

99 only-zero(x) := forall y. (x != y or y = zero).
100 zero-only(x) :- only-zero(x).

```

The frontend form uses `|-` for a real Proflog clause and `:=` for a frontend-only abbreviation:

```

101 (def peano-language
102   (pf/language
103     (constants zero)
104     (functions (s 1))
105     (relations (zero-only 1))))
106
107
108 (def zero-only-program
109   (pf/proflog peano-language
110     (:= (only-zero x)
111        (forall [y]
112          (or (!= x y)
113              (= y zero))))
114     (|- (zero-only x)
115        (only-zero x))))
116

```

After macro expansion and compilation, the kernel sees tagged formula data. The simple query `p(a)` descends schematically to:

```

117 (pos (app p (app a)))
118

```

The inlined `zero-only/1` body descends schematically to:

```

119 (forall
120   (tie y
121     (or
122       (neq (var x) (var y))
123       (eq (var y) (app zero))))))
124

```

The public query API then probes the kernel in two directions:

```

125 (query/query-succeeds program query n fuel)
126 (query/query-fails   program query n fuel)
127

```

`query-succeeds` asks the kernel to close the negated query; `query-fails` asks it to close the query itself. Open-answer examples use `pf/run`, which binds visible answer variables in the style of miniKanren's `run`, translates the query to the same backend formula, and exports bindings from the proof-backed answer layer.

2.3 Fitting's Reference Programs

Fitting's first reference program defines even and odd natural numbers over the language with constant `zero` and unary successor `s`:

```

134 even(x) :- x = zero
135          or exists y. (x = s(y) and odd(y)).
136
137 odd(x) :- forall y. (even(y) -> x != y).
138

```

The implemented frontend form is:

```

139 (pf/proflog p1-language
140   (|- (even x)
141      (or (= x zero)
142          (exists y. (x = s(y) and odd(y))))))
143

```

```

148     (exists [y]
149       (and (= x (s y))
150         (odd y))))
151   (|- (odd x)
152     (forall [y]
153       (implies (even y)
154         (!= x y)))))

```

The second reference program is a one-or-two-token Nim game. A position wins when there is a legal move to a position that does not win:

```

157 win(x) :- exists y.
158         ((x = s(y) or x = s(s(y))) and not win(y)).

```

The implementation keeps the move test inlined, as in Fitting's presentation:

```

161 (pf/proflog p2-language
162   (|- (win x)
163     (exists [y]
164       (and (or (= x (s y))
165         (= x (s (s y))))
166         (not (win y)))))

```

Table 1 summarizes the focused Fitting suite results. It covers forward success, forward refutation, answer synthesis, partial synthesis, and unresolved classification.

Table 1. Representative Fitting-program outcomes

Case	Mode	Outcome
even(0)	forward success	succeeds
odd(s(0))	forward success	succeeds
odd(0)	forward refutation	fails
win(4)	forward success	succeeds
win(3)	forward refutation	fails
move(1,0)	auxiliary forward success	succeeds
move(0,1)	auxiliary forward refutation	fails
factored win(1)	status classification	unresolved
append(x,y,[a,b,c])	answer synthesis	target found
reverse([a,b,c,a],cons(a,r))	partial synthesis	target found

A representative full output row for Fitting's first program is:

```

186 [{:description
187   "P1 proves odd(s(0)) through the forall-based odd clause",
188   :family :p1,
189   :outcome :succeeds,
190   :classification nil,
191   :max-fuel nil,
192   :id :p1-odd-1-succeeds,
193   :kind :query,
194   :proof-count 1,
195   :expected :succeeds,

```

```

197 :proof-steps
198 [neg-call
199   witness
200   conj
201   eq-step
202   par-bind
203   pos-call
204   split
205   free-close
206   witness
207   conj
208   eq-step
209   decompose
210   args
211   par-bind
212   pos-call
213   univ
214   split
215   neg-call-guarded-alt
216   guarded-alt
217   guarded-neg-alt-saturated
218   guarded-scope-done
219   guard-eq
220   eq-bind
221   guard-saturation-done
222   guarded-call-seq-done
223   guarded-seq-done
224   refl-close],
225 :proof-root neg-call}]

```

Most tags in this trace are ordinary tableau or equality work, such as witness, conj, split, univ, eq-step, and refl-close. The tags that are specific to Proflog relative to alphaLeanTAP are the procedure-call and guarded-clause steps: neg-call, pos-call, neg-call-guarded-alt, guarded-alt, and the guarded-* saturation and sequence tags. These are the proof-trace footprints of Fitting's Procedure Call rule.

The factored Nim case is included because it demonstrates that the evaluator is not merely computing the Nim recurrence in the host language. If the program introduces an auxiliary relation

```

234 win(x) :- exists y. (move(x, y) and not win(y)).
235 move(x, y) :- x = s(y) or x = s(s(y)).

```

then move/2 itself is decidable on ground calls, but win(1) remains unresolved in the tested kernel setting. This is the operational boundary Fitting warned about: a logically tempting refactoring changes how procedure calls are admitted and discharged. In the inlined program, the two move alternatives remain ordinary equality and disjunction work inside the subsidiary tableau opened for win/1. In the factored program, the same alternatives sit behind a second procedure boundary. Proflog's Procedure Call rule is not a general unfold/fold equivalence principle; it admits calls when a saved literal and the current branch state make the corresponding clause body usable. Moving formula structure into a new relation can therefore alter which subsidiary tableaux are available at

the moment a branch needs to close. The example is small, but it exposes the larger point: Proflog programs are not only extensional formula collections, but operational tableau objects.

2.4 Proof Profiles

Fitting emphasizes that Proflog gains expressiveness by using tableaux as the programming substrate, but that this substrate also inherits the search costs of first-order proof. The implementation therefore includes proof profiles: small, proof-producing dispatch layers for fragments where a specialized kernel can close a branch without changing the meaning of a successful proof. The current profiles recognize propositional formulae, equality-free first-order formulae, and equality-dominated finite fragments used by several paper examples.

Profiles are not a separate evaluator for Proflog source programs. They are entered only after the surrounding branch has been isolated from active procedure calls and the relevant equality state, and their subproof is recorded under an explicit profile tag in the proof trace. This keeps the main Procedure Call semantics intact while allowing the implementation to reuse Fitting's recognition that different fragments of the logic have different operational envelopes. The same architecture can be reused to extend and experiment with future Proflog implementations: a profile can be added for a well-understood fragment, tested as a proof-producing branch rule, and removed without changing the surface language.

3 DISCUSSION

Proflog is of interest because it gives another concrete point in the space of connections between logic and computation. The current implementation demonstrates that the core operational semantics of the tableau proof procedure, augmented with the Procedure Call rule, can be defined as a pure relation in the miniKanren style. Proof search, program evaluation, and proof-trace construction are furthermore extensible for future experimentation via the profile architecture.

The implementation is faithful to Fitting's specification in both its strengths and its limitations. Fitting's semantics require query failure to be proved, rather than assumed by search exhaustion. Undefined and unresolved cases are first-class outcomes. Refactorings that would be harmless in a purely extensional reading can change operational behavior when they introduce auxiliary procedure calls.

Performance remains an area for future work: recursive examples remain search-control frontiers, and several successful answer and partial-synthesis tests are intentionally kept out of the default fast path. However, a gap in the literature has been covered: Fitting's theoretical Proflog can now be considered mechanized as a proof-producing miniKanren-style kernel, and its non-trivial examples can be evaluated through that kernel with correctness results visible at the proof boundary.

REFERENCES

- [1] Nada Amin. 2026. leanTAP: A Theorem Prover for First-Order Classical Logic. <https://github.com/namin/leanTAP>. Clojure/core.logic.nominal implementation reference for leanTAP and alphaleanTAP. Accessed 2026-05-18.
- [2] William E. Byrd. 2009. *Relational Programming in miniKanren: Techniques, Applications, and Implementations*. Ph. D. Dissertation. Indiana University. <https://hdl.handle.net/2022/8777>
- [3] Melvin Fitting. 1994. Tableaux for Logic Programming. *Journal of Automated Reasoning* 13, 2 (1994), 175–188. <https://melvinfitting.org/bookspapers/pdf/papers/LPTableaus.pdf>
- [4] Joseph P. Near, William E. Byrd, and Daniel P. Friedman. 2008. alphaleanTAP: A Declarative Theorem Prover for First-Order Classical Logic. In *Logic Programming: Proceedings of the 24th International Conference on Logic Programming (Lecture Notes in Computer Science, Vol. 5366)*. Springer, Udine, Italy, 238–252. <https://people.csail.mit.edu/jnear/papers/alphatap.pdf>
- [5] David Nolen and contributors. 2026. core.logic. <https://github.com/clojure/core.logic>. Clojure logic programming library. Accessed 2026-05-18.